Task 1:

```
.CODE
main PROC
        mov ecx, arraySize
        mov esi, OFFSET array
inputLoop:
        mov edx, OFFSET msg1
        call WriteString
        call ReadInt
        mov [esi], eax
        add esi, 4
        call Crlf
        loop inputLoop

        ;Display
        mov edx, OFFSET msg
        call WriteString
        call Crlf
        mov ecx, arraySize
        mov esi, OFFSET array
displayArray:
        mov eax, [esi]
        call WriteInt
        mov edx, OFFSET space
        call WriteString
        add esi, 4
        loop displayArray
        call Crlf

        ;Ascending Order
        mov ecx, arraySize
        dec ecx
outerLoopAsc:
        mov esi, OFFSET array
        mov edx, arraySize
        dec edx
innerLoopAsc:
        mov eax, [esi]
        mov ebx, [esi + 4]
        cmp eax, ebx
        jbe noSwapAsc

        ;Swap
        mov [esi], ebx
```

```
        mov [esi + 4], eax
noSwapAsc:
        add esi, 4
        dec edx
        jnz innerLoopAsc
        loop outerLoopAsc

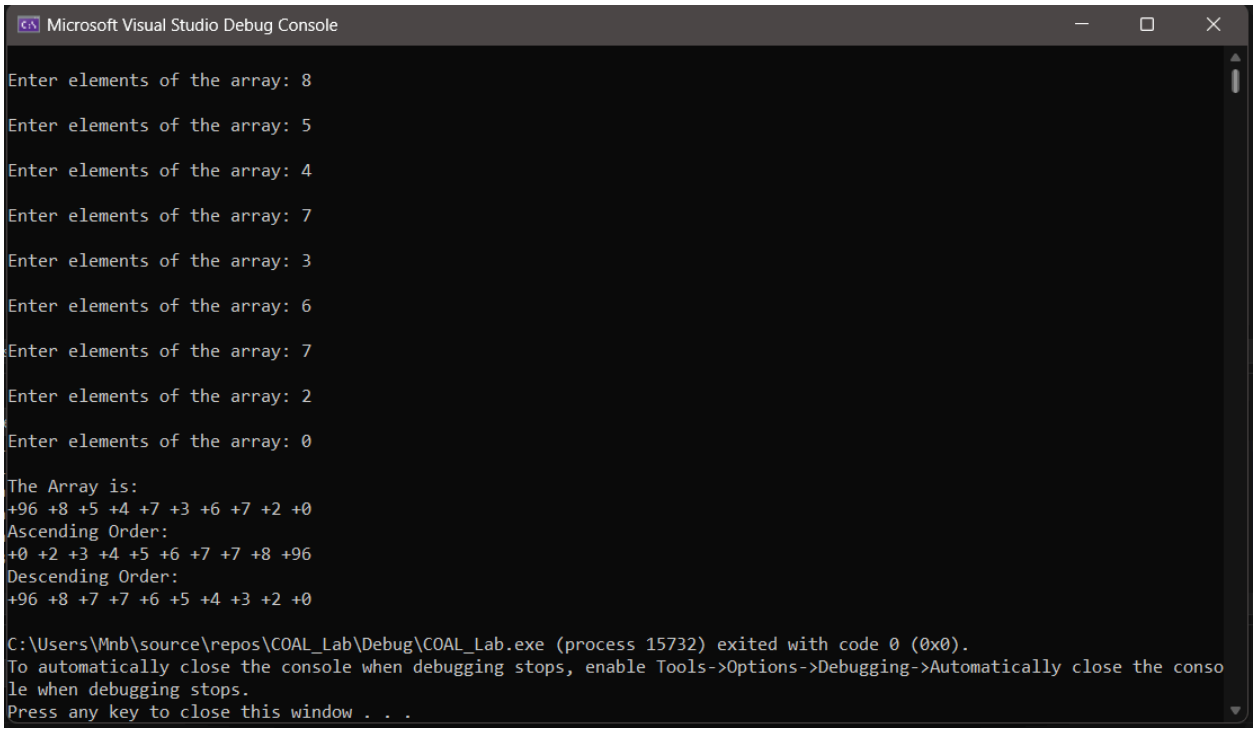
        ;Display
        mov edx, OFFSET msg2
        call WriteString
        call Crlf
        mov ecx, arraySize
        mov esi, OFFSET array
displaySorted:
        mov eax, [esi]
        call WriteInt
        mov edx, OFFSET space
        call WriteString
        add esi, 4
        loop displaySorted
        call Crlf

        ;Descending Order
        mov ecx, arraySize
        dec ecx
outerLoopDesc:
        mov esi, OFFSET array
        mov edx, arraySize
        dec edx
innerLoopDesc:
        mov eax, [esi]
        mov ebx, [esi + 4]
        cmp eax, ebx
        jae noSwapDesc

        ;Swap
        mov [esi], ebx
        mov [esi + 4], eax
noSwapDesc:
        add esi, 4
        dec edx
        jnz innerLoopDesc
        loop outerLoopDesc
```

```
        ;Display
        mov edx, OFFSET msg3
        call WriteString
        call Crlf
        mov ecx, arraySize
        mov esi, OFFSET array
displaySortedDesc:
        mov eax, [esi]
        call WriteInt
        mov edx, OFFSET space
        call WriteString
        add esi, 4
        loop displaySortedDesc
        call Crlf

        exit
main ENDP
END main
```



Task 2:

```
        .CODE
main PROC

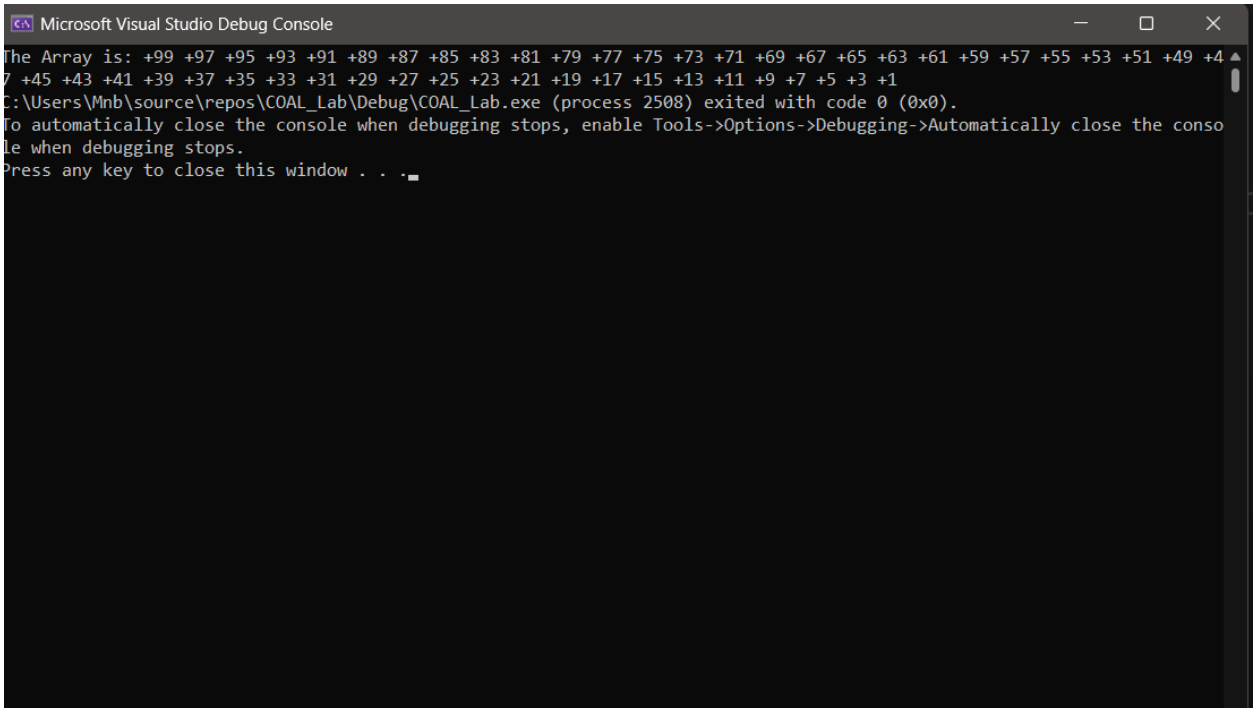
        mov ecx, 100
        inc ecx
```

```
        mov edi, offset array
        L1:
        dec ecx
        cmp ecx, 0
        JE end_loop
        mov eax, ecx
        mov edx,0
        mov ebx,2
        div ebx
        cmp edx,0
        JE L1
        mov [edi],ecx
        add edi, 4
        cmp ecx,0
        JNE L1

        end_loop:
        mov edx,offset msg
        call writestring
        mov esi, offset array
        mov ecx, arraySize
        mov edx, offset space
        display_loop:
        mov eax,[esi]
        add esi,4
        call writeint
        call writestring

        loop display_loop

        exit
main ENDP
END main
```



Task 3:

```
.CODE
main PROC

        mov edx, offset msg
        call writestring
        call readint
        mov n1,eax

        mov edx, offset msg1
        call writestring
        call readint
        mov n2,eax


        mov eax, n1
        mov ebx, n2
        mov ecx, arraySize
        mov esi,offset array
        sub esi,4
        mov edi,offset ans
        L1:
        cmp ecx,0
        JE L1_2
        add esi,4
```

```
        dec ecx
        cmp [esi],eax
        JNE L1
        mov temp1, 1

        L1_2:

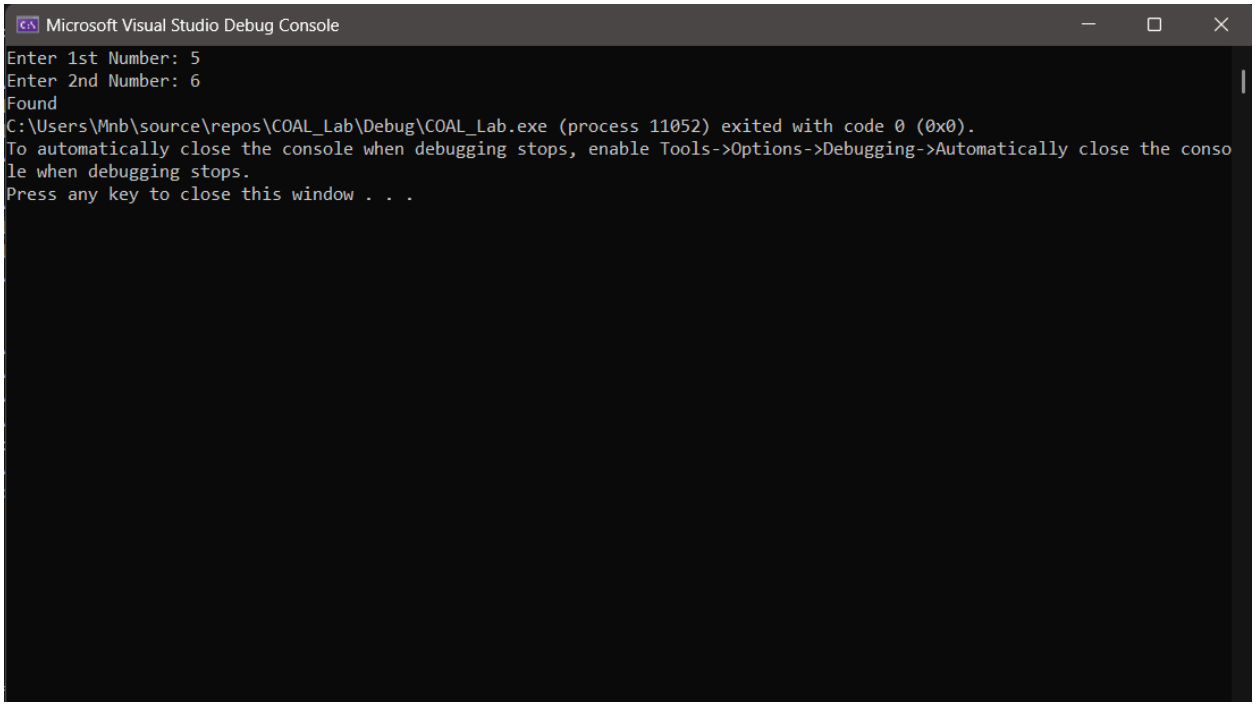
        mov ecx, arraySize
        mov esi,offset array
        sub esi,4

        L2:
        cmp ecx,0
        JE comp
        add esi,4
        dec ecx
        cmp [esi], ebx
        JNE L2
        mov temp2, 1

        comp:
        mov eax,temp1
        mov ebx,temp2
        and eax,ebx
        JZ not_found
        mov edx,offset foundmsg
        call writestring
        jmp end1
        not_found:
        mov edx, offset notfoundmsg
        call writestring

        end1:


        exit
main ENDP
END main
```



Task 4:

```
.CODE
main PROC

        mov edx, offset prompt1
        call writestring
        call readint
        mov n1,eax

        mov edx, offset prompt2
        call writestring
        call readint
        mov n2,eax

        mov edx, offset prompt3
        call writestring
        call readint
        mov n3,eax

        mov edx, offset msg
        mov eax, n1
        mov ebx, n2
        mov ecx, n3

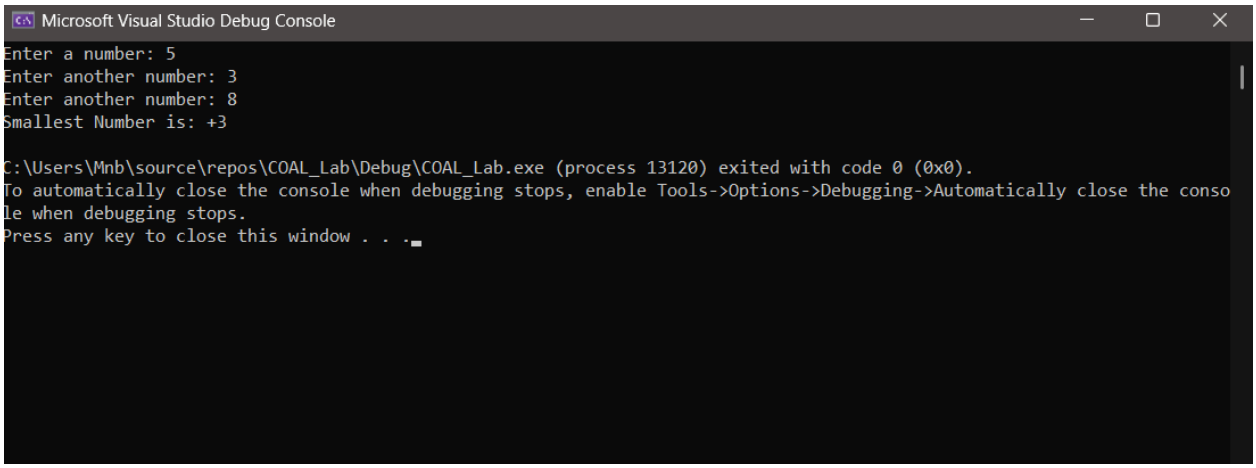
        cmp eax, ebx
        jbe compThree
```

```
        mov eax,ebx

        compThree:
        cmp eax,ecx
        jbe print
        mov eax,ecx

        print:
        call writestring
        call writeint
        call crlf

        exit
main ENDP
END main
```



Task 5:

```
.CODE
main PROC

        mov edx, offset prompt
        call writestring
        call readint
        mov n1,eax

        cmp eax, 1
        jbe not_prime

        mov eax,n1
        cmp eax,2
        je prime
```

```
        mov ecx, n1
        dec ecx
        mov edx, 0
        mov eax, n1
        mov ebx, 2
        div ebx
        mov eax,n1

        main_loop:
        mov edx, 0
        div ebx
        cmp edx,0
        je not_prime

        inc ebx
        cmp ebx,ecx ;ecx is n-1
        jb main_loop

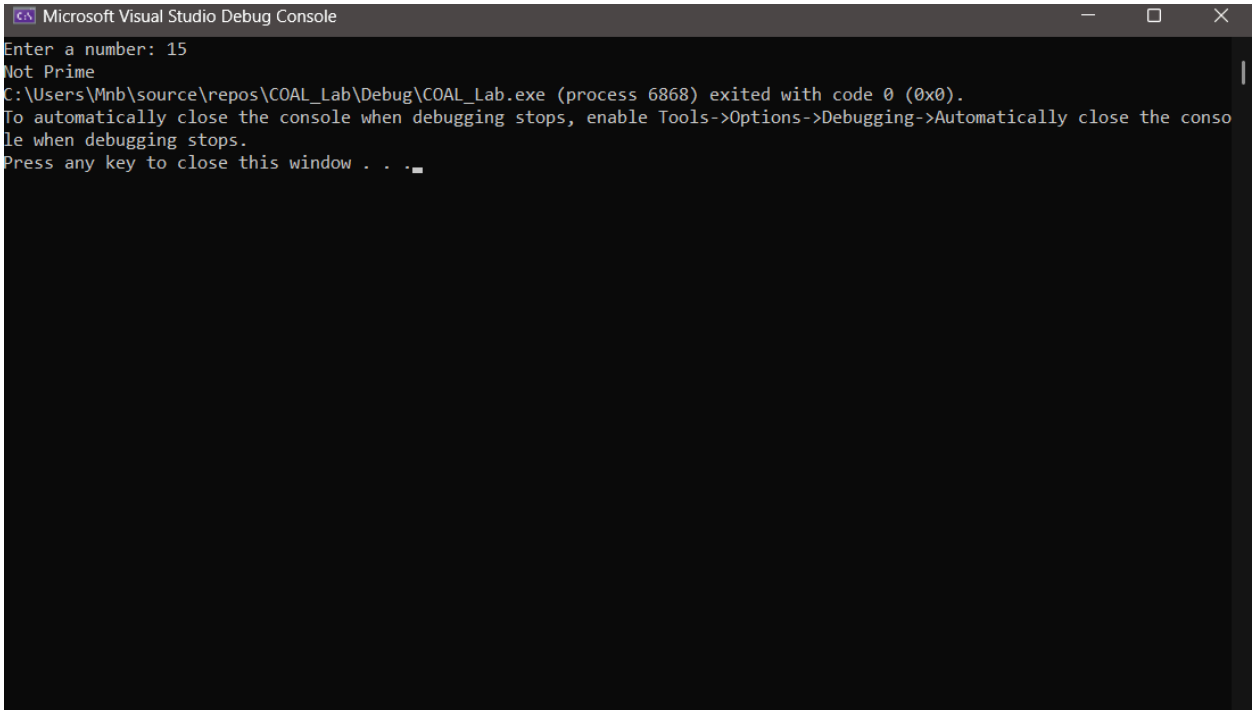


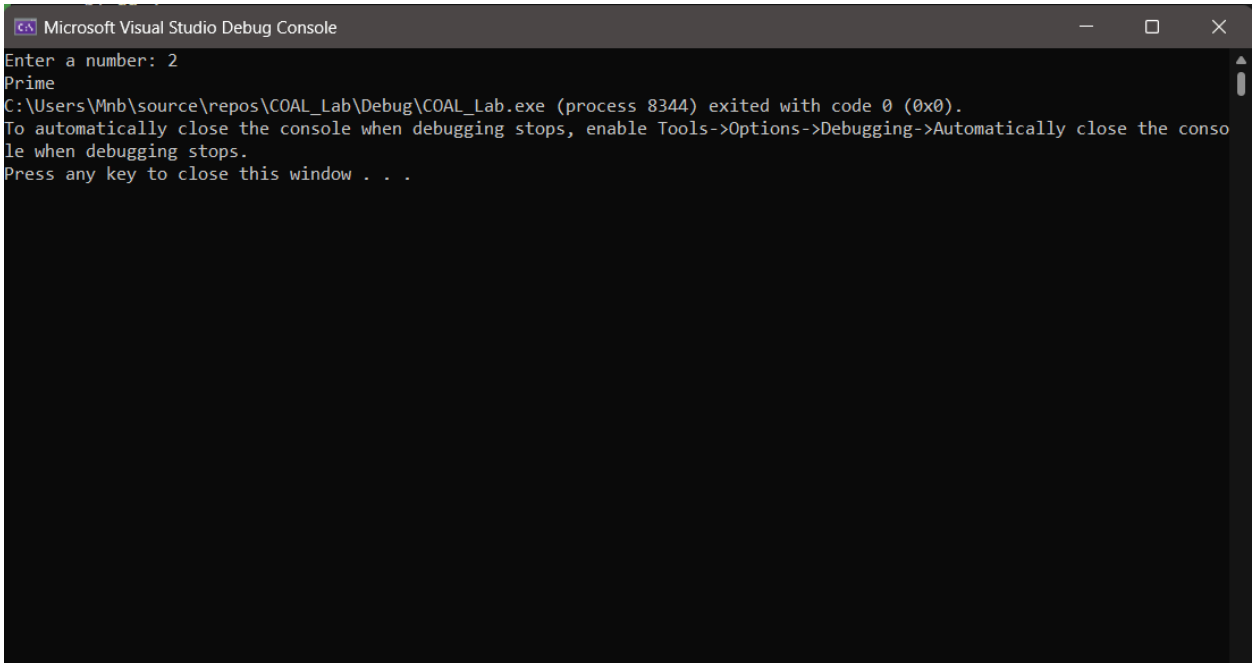
        prime:
        mov edx, offset msg1
        call writestring
        jmp end_prog

        not_prime:
        mov edx, offset msg2
        call writestring
        jmp end_prog

        end_prog:

        exit
main ENDP
END main
```





Task 6:

```
.CODE
main PROC

        mov ecx, max
        mov edx, offset msg
        outer_loop:
```

```
        mov eax,max
        inner_loop:
        call writestring
        dec eax
        cmp eax,0
        jnz inner_loop

        call crlf
        loop outer_loop

        exit
main ENDP
END main
```

